

Nesso N1 Gesture Recognition

Technical Project Report

Hardware	Arduino Nesso N1 ESP32-C6 BMI270 6-axis IMU
Gesture Classes	circle / shake / swipe_h / swipe_v / tap
ML Platform	Edge Impulse (migrated from TensorFlow Lite / Google Colab)
Current Accuracy	61% — Iteration 2 in progress
Author	Concordia University — Computer Engineering
Date	May 2026

Abstract

This report documents the end-to-end development of a real-time gesture recognition system deployed on the Arduino Nesso N1 embedded board. The project progressed through two major iterations: an initial TensorFlow Lite pipeline achieving 35% validation accuracy after data cleaning, followed by migration to Edge Impulse achieving 61%. The report covers system architecture, data collection methodology, signal processing design decisions, observed failure modes, and the engineering improvements applied at each iteration.

1. Introduction

Gesture recognition on embedded hardware represents a core challenge in Edge AI: inference must run in real time on a microcontroller with limited memory, compute, and power budget, using only raw IMU sensor data as input.

This project implements a five-class gesture classifier on the Arduino Nesso N1 board, which integrates an ESP32-C6 microcontroller, a BMI270 6-axis IMU, and a 240x135 ST7789 TFT display. The classifier runs entirely on-device with no cloud dependency, recognising circle, shake, swipe_h, swipe_v, and tap gestures in real time.

1.1 Project Objectives

- Collect high-quality IMU gesture data using an auto-trigger collection pipeline
- Train and quantize an int8 classifier suitable for deployment on ESP32-C6
- Achieve reliable real-time inference with visual feedback on the built-in display
- Document all iterations, failure modes, and improvements for portfolio purposes

2. System Architecture

2.1 Hardware Platform

Component	Specification	Role
MCU	ESP32-C6 @ 160 MHz (RISC-V)	Inference + display + control
IMU	BMI270 — 6-axis (accel + gyro)	Gesture signal source
Display	ST7789 240×135 TFT via SPI	Real-time feedback
Buzzer	BEEP_PIN (GPIO 11), PWM	Haptic/audio confirmation
Buttons	KEY1 (pause/resume), KEY2 (label cycle)	User interaction
Interface	USB-C serial @ 115200 baud	Data streaming to PC

2.2 System Pipeline

The full pipeline from data collection to live inference is shown in Figure 1. All inference runs on-device with no cloud dependency post-deployment.



Figure 1. End-to-end system pipeline from collection to inference.

2.3 Software Stack

Layer	Tool / Library	Purpose
Data Collection	gesture_collect.ino + collect.py	Auto-trigger IMU stream → CSV
ML Training	Edge Impulse Studio	DSP, training, int8 quantization, export
Inference Runtime	Edge Impulse Arduino SDK	run_classifier() on ESP32-C6
Display Driver	NessoDisplay (LovyanGFX / M5GFX)	ST7789 TFT rendering
Serial Capture	Python 3 + pyserial	PC-side CSV writer
Previous Pipeline	TensorFlow Lite + Google Colab	Replaced — Phase 1 & 2

3. Data Collection Methodology

3.1 Gesture Classes

Five gesture classes were defined, chosen to provide distinct IMU signatures across the accelerometer and gyroscope axes. Horizontal and vertical swipes are treated as separate classes because they produce dominant acceleration on orthogonal axes (X vs. Y), and merging them was found to confuse the classifier.

Gesture	Primary Axis Signal	Threshold Offset	Window	Target Captu
circle	gyroZ (angular velocity)	+1.5 g	1.5 s / 15 samples	100
shake	accel XYZ (periodic burst)	+1.5 g	2.0 s / 20 samples	100
swipe_h	accelX dominant	+1.5 g	0.8 s / 8 samples	100
swipe_v	accelY dominant	+1.5 g	0.8 s / 8 samples	100
tap	accel spike (all axes)	+2.5 g	0.5 s / 5 samples	100

Table 1. Gesture class specifications and collection parameters.

3.2 Auto-Trigger Collection Architecture

Data collection uses an auto-trigger mechanism rather than manual button-press recording. This was a deliberate design decision to improve data quality: pressing a button simultaneously with performing a gesture introduces noise from the press motion itself and delays the recording start relative to gesture onset.

- Board samples IMU at 10 Hz (delay(100))
- Rolling baseline continuously tracks ambient acceleration magnitude
- Trigger fires when: magnitude > rollingBaseline + gestureThreshold
- Pre-trigger circular buffer (5 samples = 500ms) captures motion wind-up before threshold crossing
- After capture window closes, peak accel is checked — if below 1.8g, capture is discarded
- 1500ms cooldown prevents return hand motion from triggering a second capture

3.3 Adaptive Threshold Design

A fixed threshold was found to produce false triggers when the ambient noise level varied (e.g. resting on a vibrating surface). The adaptive design computes a rolling baseline:

```
rollingBaseline = rollingBaseline × 0.95 + magnitude × 0.05
```

The gesture-specific offset is then added on top of this baseline. Tap uses a higher offset (2.5g) because it produces a sharp, high-magnitude spike that must be distinguished from incidental bumps. All other gestures use 1.5g.

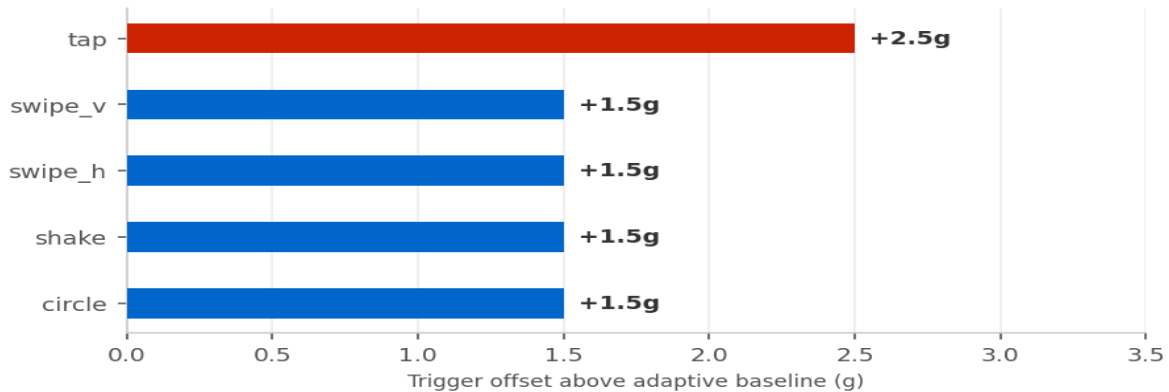


Figure 2. Per-gesture trigger thresholds above adaptive baseline.

3.4 Collection Protocol

Each capture session records one clean, isolated gesture motion. The operator selects the gesture label using KEY2 on the board (which cycles through all five classes displayed on screen), then performs the motion. The board automatically captures, validates, and streams the sample to the PC.

- Both directions collected for swipe_h (left→right and right→left) for directional invariance
- Both directions collected for circle (clockwise and anti-clockwise)
- Shake: varied intensity — both gentle and vigorous captures
- Tap: single clean tap per capture, board held stable before and after
- Target: 100 captures per class (500 total captures)

4. Accuracy Progression

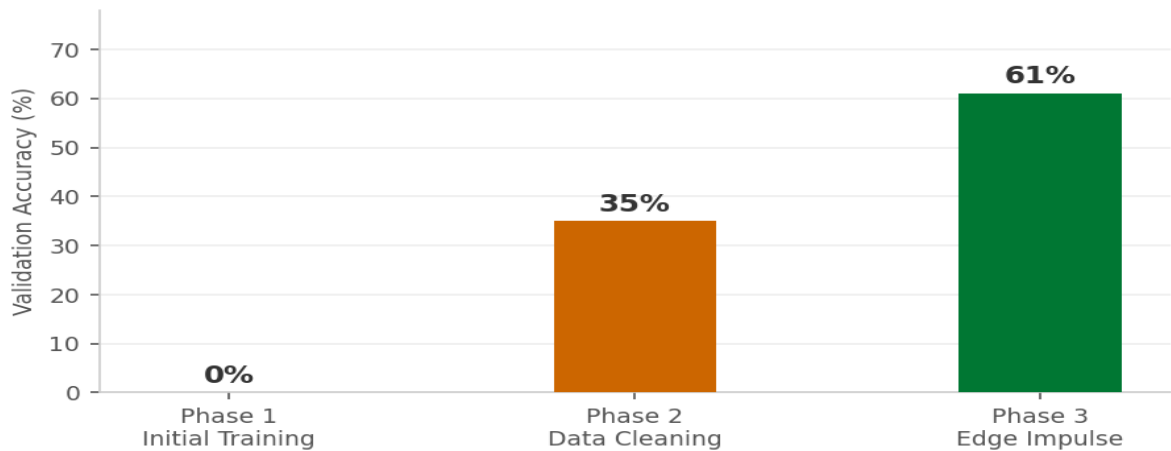


Figure 3. Validation accuracy across three development phases.

Phase	Focus	Outcome	Tool
Phase 1	Initial data collection and training	0% accuracy — fundamental data quality failure	TensorFlow Lite / Colab
Phase 2	Data cleaning and preprocessing	~35% accuracy — baseline established	TensorFlow Lite / Colab
Phase 3	Platform migration to Edge Impulse	61% accuracy — production pipeline	Edge Impulse

Table 2. Development phase summary.

4.1 Phase 1 — Initial Training (0% Accuracy)

Initial data was collected using a manual serial monitor approach with no threshold gating. The resulting dataset contained significant noise: still-state samples labelled as gestures, inconsistent window lengths, and mixed-direction captures under a single label. The model trained to 0% validation accuracy, indicating it had learned no discriminative features.

4.2 Phase 2 — Data Cleaning (35% Accuracy)

Preprocessing improvements included outlier removal, consistent window sizing, and StandardScaler normalisation. A simple MLP architecture (32→64→32→softmax) was trained on the cleaned dataset achieving approximately 35% validation accuracy. This established a functional baseline but remained insufficient for reliable real-time inference.

4.3 Phase 3 — Edge Impulse Migration (61% Accuracy)

Migration to Edge Impulse provided DSP preprocessing blocks, a visual feature explorer for verifying class separation, and a streamlined int8 quantization pipeline. The platform achieved 61% validation accuracy on the same gesture set. Further improvement is expected with the expanded 100-capture-per-class dataset currently being collected using the auto-trigger pipeline.

5. Observed Issues and Engineering Fixes

The following table documents failure modes observed during development, their root causes, and the engineering solutions applied.

Observed Issue	Root Cause	Fix Applied
Still motion classified as shake/tap	Fixed threshold too low; no baseline calibration	Adaptive rolling baseline with per-gesture offsets
Swipe immediately followed by tap detected	Re-trigger hand motion re-triggered inference	500ms cooldown post-detection
0% accuracy in Phase 1	Noisy, inconsistent training data with no quality gate	Auto-gate pipeline + weak capture rejection (discarded)
Model output all -0.0% confidence	Single IMU sample fed to model expecting condensed input	Condensed input pipeline; proper int8 quantization
Output tensor type mismatch	Reading float values from int8 output tensor	Proper dequantization: float = (int8 - zero_pt) × scale
False triggers when picking up board	No motion gate; threshold crossed by handling motion	Motion gate + calibration window on boot
swipe_h and swipe_v confused	Both labelled as single "swipe" class despite orthogonal directions	Splitting into two classes; separate collection

Table 3. Observed issues, root causes, and fixes.

6. Engineering Improvements — Iteration 2

The following improvements were implemented between Phase 1 and the current state of the project. Each improvement was motivated by a specific observed failure mode documented in Section 5.

Improvement	Description	Motivation
Pre-trigger Circular Buffer	5 samples (500ms) buffered before trigger fires; flushed on motion capture start	Captured motion capture start missed when recording
Adaptive Threshold	Rolling baseline: baseline = baseline×0.95 + magnitude×0.05	Fixed 0.05 offset problem of false triggers in varying
Weak Capture Rejection	Captures discarded if peak accel < 1.8g; board shows LED when valid	Prevents weak, noisy captures from entering training
Per-Gesture Thresholds	Tap: +2.5g offset. All others: +1.5g offset above baseline	Tap's sharp impact signature requires higher threshold
Anti-Spam Cooldown	500ms IMU ignore window after each detection in gesture detection	Baseline detection was re-triggering inference in
Edge Impulse Migration	Full pipeline migrated from TFLite/Colab to Edge Impulse Studio	Better IDE blocks, feature explorer, confusion matrix
swipe_h / swipe_v Split	Single swipe class replaced with two directional classes	X-axis and Y-axis swipe signals are orthogonal; better
Buzzer Volume Reduction	PWM duty cycle reduced from 128 (50%) to 64 (25%)	Audible confirmation without disrupting collection

Table 4. Engineering improvements implemented in Iteration 2.

7. Inference Pipeline

7.1 Detection Sketch Architecture

The `gesture_detect.ino` sketch implements the following detection loop at 10 Hz:

- On boot: 3-second calibration window establishes rolling baseline (30 samples)
- Each loop: IMU sampled, adaptive baseline updated, motion gate checked
- If magnitude exceeds gate: sample buffered into Edge Impulse input frame
- When frame full: `run_classifier()` invoked, result checked against 60% confidence floor
- If confidence $\geq 60\%$: gesture displayed on TFT, single beep fired, 500ms cooldown starts
- KEY1 single press: pauses inference (PAUSED screen shown)
- KEY1 double press (within 400ms): resumes inference

7.2 Display States

State	Display	Trigger
Calibrating	Yellow progress bar — "Calibrating... Hold still!"	On boot for 3 seconds
Waiting	Grey text — "Waiting..."	No motion above gate threshold
Detected	Green gesture name + confidence %	Inference result $\geq 60\%$ confidence
Paused	Orange — "PAUSED" + resume hint	KEY1 single press
Error	Red screen + error message	IMU fail / model load fail / alloc fail

Table 5. Display state machine.

8. Next Steps and Future Work

8.1 Immediate

- Complete 100-capture-per-class data collection using auto-trigger pipeline
- Upload `gesture_data.csv` to Edge Impulse; verify class separation in feature explorer
- Train model; review per-class accuracy and confusion matrix
- Export int8 Arduino library; flash `gesture_detect.ino`; tune confidence threshold

8.2 Short Term

- Target $>75\%$ validation accuracy with 500-sample dataset
- Add confusion matrix and per-class precision/recall to this report
- Characterise latency: measure inference time per classification on ESP32-C6

8.3 Long Term

- Expand gesture vocabulary (rotation, figure-eight, push/pull)

- Implement deep sleep / IMU wake-on-motion for battery life optimization
- Publish project to GitHub as documented Edge AI / TinyML portfolio piece
- Target embedded AI roles at NVIDIA, Qualcomm, Tesla — project demonstrates full embedded ML pipeline from data collection to on-device inference

A. File Listing

File	Description
gesture_collect/gesture_collect.ino	Auto-trigger IMU data collection sketch. KEY2 cycles label, motion triggers c
gesture_detect/gesture_detect.ino	Real-time inference sketch. Adaptive threshold, Edge Impulse SDK, KEY1 p
gesture_detect/model.h	Edge Impulse exported model header (generated post-training — not yet fina
gesture_detect/scaler.h	Feature normalization parameters (generated post-training — not yet final).
collect.py	Python serial collector. Reads LABEL/END/DISCARD signals, writes gesture
gesture_data.csv	Training dataset. Columns: accelX, accelY, accelZ, gyroX, gyroY, gyroZ, lab